SCOPE: A Real-Time Natural Language Camera Agent at the Edge

A Sim-to-Real Benchmark and Analysis of Open-Source Vision and Language Agents for PTZ Camera Tasks

Nikolaj Hindsbo
nh@armada.ai
Armada AI
Bellevue, Washington, USA

Sina Ehsani
se@armada.ai
Armada AI
Bellevue, Washington, USA

Pragyana Mishra
pm@armada.ai
Armada AI
Bellevue, Washington, USA

Abstract

Deploying language-driven agents in robotics requires evaluations that reflect real-world task demands: natural-language instructions with reproducible outcomes. Such agents must connect language models (LMs) to callable perception and control tools, and be assessed using deployment-critical metrics including latency, accuracy, and error modes. We present SCOPE (Simulation and Camera Operations for Perception and Evaluation), a modular agent for natural-language, open-vocabulary pan–tilt–zoom (PTZ) camera control and visual scene understanding, designed explicitly for edge deployment. SCOPE operates both in a Blender-based simulation environment and on a physical PTZ camera, executing all perception, planning, and control locally at the deployment site using edge-accessible compute.

We introduce a Blender-based agent environment that exposes realistic PTZ control affordances and enables reproducible, language-driven tasks aligned with real-world camera operation. Using this environment, we release a 536-task benchmark spanning QA, single- and multi-step commands, counting, spatial reasoning, descriptions, and optical character recognition. Execution traces are combined with an LM-as-Judge to evaluate latency, accuracy, and error modes.

Across 19 model combinations pairing Qwen3 Small Language Models (SLMs) with Moondream and Qwen vision–language models (VLMs), the best-performing configuration (Qwen3-30B-A3B × Moondream3) achieves 73.8% task success. Planner choice drives large accuracy swings (6–12%), while VLM selection contributes an additional ≈ 3% variation on average; however, among the strongest SLMs, perception quality remains the dominant bottleneck. Beyond model choice, we find that architectural decisions materially affect performance and deployability: Mixture-of-Experts architectures—on both the planner and perception side—consistently match or exceed dense models in our evaluation while operating at inference latencies and memory footprints comparable to much smaller networks, and quantization further reduces cost with negligible accuracy degradation. Together, these results identify a practical, sim-to-real–validated, edge-feasible design point for real-time, language-driven PTZ control.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
HRI '26, Edinburgh, Scotland, UK
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
https://doi.org/10.1145/3757279.3785641

CCS Concepts

• **Computer systems organization** → *Robotics*; *Real-time system architecture*; • **Computing methodologies** → **Intelligent agents**.

Keywords

PTZ, agentic control, small language models, vision–language models, edge compute, simulation, human–robot interaction

ACM Reference Format:
Nikolaj Hindsbo, Sina Ehsani, and Pragyana Mishra. 2026. SCOPE: A Real-Time Natural Language Camera Agent at the Edge: A Sim-to-Real Benchmark and Analysis of Open-Source Vision and Language Agents for PTZ Camera Tasks. In *Proceedings of the ACM/IEEE International Conference on Human-Robot Interaction (HRI '26), March 16–19, 2026, Edinburgh, Scotland, UK*. ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3757279.3785641

1 Introduction

Operating a pan–tilt–zoom (PTZ) camera from natural language sits at the intersection of perception, planning, and precise actuation under strict latency and compute constraints. A seemingly simple instruction such as "go to the highway preset, then pan right in 15° steps until you see at least six cones" expands into a non-trivial feedback loop that couples perception, reasoning, and motion. Figure 1 shows this process unfolding in a real end-to-end deployment of our language-driven PTZ agent. Executing such tasks autonomously on edge hardware is challenging because success depends not on isolated perception or reasoning, but on reliable coordination of language, vision, and actuation under tight memory and latency budgets.

Recent benchmarks demonstrate rapid progress in general reasoning [10], multimodal understanding [22], and tool invocation [15]. However, these evaluations remain indirect for embodied, language-guided control. Most do not measure closed-loop behavior, struggle to make sim-to-real claims, and do not capture deployment-critical constraints such as latency, cost, and error modes on edge hardware. As a result, it remains difficult to predict how a language-driven agent will behave—and fail—when deployed in real robotic control loops.

To address this gap, we present Simulation and Camera Operations for Perception and Evaluation (SCOPE), a Blender-based framework and benchmark for executable, closed-loop evaluation of language-driven PTZ camera agents. By exposing an identical PTZ action space and observation loop in both simulation and on a physical camera, SCOPE enables faithful sim-to-real transfer and evaluation for PTZ agents.

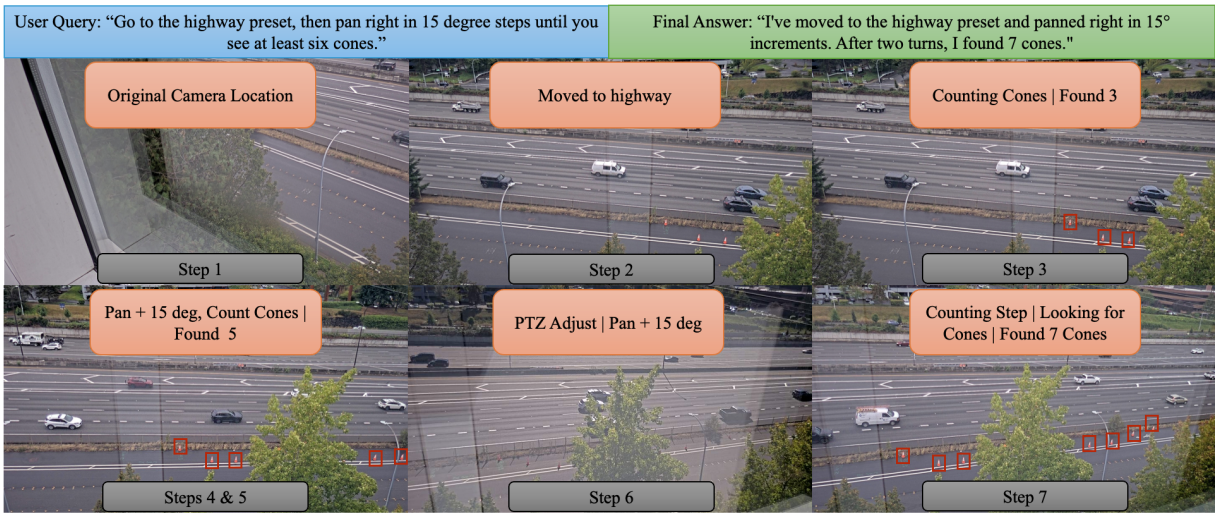

Figure 1: Example of SCOPE executing a multi-step language-guided PTZ task our physical camera. The user requests: "Go to the highway preset, then pan right in 15° steps until you see at least six cones." The agent moves to the preset, performs incremental panning, and invokes the VLM to count cones after each step. For visualization clarity in the paper, cones are shown with bounding boxes rather than the point-based indicators returned by the system at runtime. Step-wise annotations summarize camera actions and intermediate counts, and the final response aggregates these observations, confirming seven cones.

Our contributions are as follows:

* We evaluate the tasks
* we present the framework
* create a benchmark

2 Related Work

In robotics, language models are increasingly used as planners that call grounded skills rather than as low-level controllers. *PaLM-SayCan* grounded open-ended instructions in robot affordances and value functions, showing that language plans become useful when constrained by what the robot can actually do [2]. *Code as Policies* showed that language models can produce code to control robots on-the-fly, demonstrating generalization while highlighting the need for tight output constraints to avoid execution errors [13]. Conversational prompting for robotics with capability schemas has also been examined (e.g., ChatGPT for robotics), emphasizing structured formats to reduce invalid commands [20]. These works underscore the potential of SLMs as high-level planners but also the need to ground decisions in actual actions to avoid hallucinations or infeasible plans.

While Large Language Model (LLM) planning is now well established, most work on language-driven active perception has focused on tasks involving mobile agents. Foundational benchmarks like Vision-and-Language Navigation [3, 19] and Embodied Question Answering [8] require an agent to physically navigate complex 3D environments to find objects or answer questions. Although this work is highly relevant, the focus on mobile navigation means these benchmarks do not address the specific challenges of controlling a static PTZ camera. Furthermore, their evaluation criteria are not centered on the sim-to-real parity of tool-calling agents or the deployment-critical metrics (e.g., latency, cost) essential for edge hardware, leaving this distinct problem domain largely under-explored.

This opportunity is unlocked by modern VLMs (e.g., Qwen2.5-VL [4], Moondream [12], and Molmo [9]), which have become increasingly useful for robotics. Beyond free-form captions, many now emit structured outputs—counts, optical character recognition (OCR), strings, attributes, and spatial primitives (points/boxes)—often encoded as JSON, making them directly consumable by planners. Grounded supervision with points/boxes and large open synthetic datasets (e.g., PixMo [9]) has reduced counting and commonsense biases and improved robustness against memorization (e.g., stock footage of clock faces or animals with extra limbs/parts that break dataset priors [12]).

Lightweight VLMs in our study also reliably emitted JSON-formatted outputs without parsing issues. Moondream, in particular, is designed for efficient deployment on commercial hardware: it runs on an NVIDIA RTX 3090 with a memory footprint of 2.45 GB and achieves up to 184 tokens/s throughput [12]. We validated this efficiency in our own evaluation; when hosted locally on an A100-80GB GPU, all lightweight VLMs produced consistent query and counting responses within 200–500 ms, supporting their suitability for real-time control loops. For clarity, we define "real-time" in this context as sustaining an approximately 2 Hz control-loop frequency ($\leq$500 ms per inference), which is sufficient for PTZ camera scanning and framing tasks. In contrast, our Blender-based simulation evaluations relied on a shared cloud cluster, where network and scheduling overheads led to longer response times.

Prior to SCOPE, we developed OPUS [6], which demonstrated a few-shot, tool-calling pipeline for language-driven PTZ control. While functional, OPUS exhibited several practical limitations that

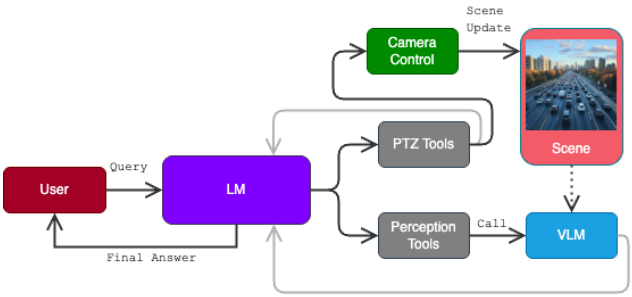

Figure 2: SCOPE architecture. The LM interleaves reasoning with PTZ actions (pan/tilt/zoom) and perception queries (counts, OCR, attributes). Results are fed back into context until a final answer is produced.

constrained extensibility and deployment. In particular, introducing new tools required task-specific data generation and fine-tuning. Furthermore, perception was handled through You Only Look Once (YOLO) [17]–style detectors, which limited the range of visual queries supported.

3 System Architecture

Figure 2 illustrates the SCOPE architecture. The system adopts a decoupled design in which a compact Small Language Model (SLM) serves as a high-level planner, interleaving reasoning with camera control and perception queries, while visual understanding is delegated to lightweight vision–language models (VLMs) exposed as callable tools. Rather than streaming image tokens into the dialogue—which can introduce hundreds to tens of thousands of tokens per frame and compound across turns [4, 12, 21]—the planner invokes perception tools on demand and reasons over concise, task-aligned outputs.

This separation reflects the current capabilities of open-source models: robust visual perception and reliable tool-based control remain difficult to achieve jointly within a single model at real-time edge latencies. By decoupling planning from perception, SCOPE achieves predictable latency, preserves modularity for independently upgrading planners and perception backbones, and improves safety and interpretability, enabling closed-loop PTZ control on local hardware.

Tool calls issued by the SLM are routed either to (i) the VLM, which returns grounded textual observations such as object counts, OCR results, or semantic attributes, or (ii) the PTZ camera API, which executes direct control commands. The PTZ path covers both viewpoint adjustments (pan, tilt, zoom) and direct outputs (e.g., screenshots), which can be appended to the agent's context without invoking perception. All tool outputs are fed back into the planner's context, enabling iterative reasoning and action until task completion.

3.1 Agent Loop and Control

At each step the agent queries the LM with the user request, history, and any tool observations. The model either terminates with a natural-language answer or issues tool calls; their outputs are appended into the dialogue state for the next step. This repeats until an answer is produced or a step cap $T_{\max}$ is reached.

Formally, SCOPE is a recurrent state machine with state $S_t = (U, H_t, O_t)$, where U denotes the fixed user query or goal, H_t the cumulative dialogue and tool-call history, and O_t the sequence of observed tool outputs. At each step t, the planner emits either a final answer or a structured tool-call sequence τ_t, which yields the tool result R_t. The system then updates its internal memory as $H_{t+1} = H_t \parallel [\tau_t, R_t]$ and observation log as $O_{t+1} = O_t \parallel [R_t]$. The process continues until a final answer is produced or the maximum reasoning horizon $T_{\max}$ is reached. This formulation captures SCOPE as a discrete-time decision process whose evolution depends on recurrent integration of actions and observations.

3.2 Capabilities and Tool Interface

The SCOPE agent controls and queries the camera exclusively through callable Python functions exposed to the SLM planner via OpenAI-compatible JSON schemas. This design enables the planner to invoke tools with explicit, structured arguments (e.g., numeric pan/tilt offsets or object descriptions).

Direct camera commands provide explicit control over the PTZ camera and its configuration:

- `ADJUST_PTZ`: Applies relative pan, tilt, and zoom adjustments (e.g., pan=+10°, tilt=+10°).
- `GO_TO_PRESET`: Moves the camera to a named, predefined position and field of view.
- `GO_HOME`: Returns the camera to its default home orientation.
- `GET_PRESETS`: Retrieves the list of available preset viewpoints.
- `TAKE_IMAGE`: Captures and saves the current camera frame.

Perception tools couple camera observations with lightweight vision–language models and return semantic information to the planner:

- `COUNT_OBJECTS`: Counts objects matching a natural-language description, over the current view or full panorama.
- `QUERY_ANSWER`: Submits the current view or panorama and a natural-language query to the VLM and returns the model response.
- `ZOOM_TO_OBJECT`: Queries the VLM for a structured bounding-box localization of a target object and maps this output to PTZ control commands.
- `TRACK_OBJECT`: Maintains focus on a target by repeatedly parsing structured VLM localizations and issuing corresponding PTZ adjustments.

3.3 Real-World Setup

Development began initially on a physical system. An AXIS M5075-G PTZ camera was wrapped behind the same tool schema from Subsection 3.2, and a Streamlit interface exposed the camera livestream, a natural-language control panel, an annotated-frame gallery, and per-turn timing breakdowns.

This real deployment was essential for validating end-to-end feasibility and for uncovering practical failure modes. In particular, we observed that agent behavior could be destabilized by small prompt edits, with errors such as incorrect tool selection or inverted control actions (e.g., panning left when asked to pan right). Such sensitivity is well documented for SLMs, which can react strongly to minor

changes in phrasing or prompt length [5]. However, progress at this stage was necessarily qualitative—assessed by whether behavior "felt right" and lacked a systematic way to compare models, prompts, or configurations.

These limitations motivated the transition to a simulation-based evaluation framework that would support deterministic replay, controlled ablations, and scalable benchmarking.

3.4 Simulation Setup

To enable repeatable evaluation and CI-style testing, we implemented the agent in simulation using Blender, an open-source 3D environment with a Python API. In Blender, the camera object can be entered directly and all intrinsic and extrinsic parameters are programmatically accessible. By fixing the camera position (x, y, z), allowing yaw and pitch rotations, and controlling focal length to adjust field of view, we recreate the operational degrees of freedom of a PTZ camera.

All tools described in Subsection 3.2 were re-implemented in this environment with identical names, parameters, and JSON schemas. From the SLM planner's perspective, the interface is unchanged; only the backend differs (physical AXIS device versus Blender). This one-to-one correspondence enables direct comparability between simulation and real deployment. Representative simulated views are shown in Figure 3.

Some real-world queries require awareness beyond a single PTZ frame—for example, surveying an entire parking lot or reasoning about objects that may be outside the current view. Naïvely stitching images from different timeframes would introduce two major issues: (i) viewpoint overlap that could lead to double-counting or miscounts, and (ii) logical inconsistencies, since scenes can change between frames.

To avoid these issues while keeping the evaluation focused, we implement a panorama tool that performs a single, deterministic sweep and stitches the resulting frames into a unified image representing the entire scene. This design allows us to evaluate two complementary capabilities: (i) the VLM's ability to interpret stitched, multi-view imagery that departs from standard single-frame inputs, and (ii) the SLM's ability to recognize when invoking a full-scene observation is necessary to answer a query.

The setup supports two complementary modes. First, one-off questions can be posed interactively inside Blender through a lightweight UI banner—useful for debugging or rapid testing. Second, scripted batch execution runs entire sets of questions sequentially across scenes, enabling large-scale evaluation or reinforcement learning experiments.

4 Benchmark Construction

4.1 Task Design and Coverage

We construct a benchmark of 536 tasks to evaluate language-driven PTZ agents under realistic operating conditions. The benchmark spans action-oriented commands and question–answering tasks grounded in visual perception and tool use, with all tasks requiring explicit viewpoint changes. Typical examples include "sweep the scene and count all cars" (Counting), "zoom in on a storefront window and read the sign" (OCR), and "navigate to a predefined view and take a snapshot" (Multi-step Command). This design captures the core challenges of PTZ control beyond static image understanding. Table 1 reports the distribution of tasks across evaluation categories.

Figure 3: Representative Blender scenes used in simulation, showing diverse urban environments with signage, occlusion, and visual variation. Each scene includes multiple fixed camera presets at different locations and viewing angles.

4.2 Environment and Viewpoint Construction

Because PTZ operation fundamentally relies on viewpoint control, we require environments in which a camera can meaningfully pan, tilt, and zoom rather than observe a static scene. We therefore construct our benchmark using four publicly available Blender scenes representing diverse urban settings with signage, occlusion, clutter, and varying environmental conditions.

Within each scene, we define multiple fixed camera presets at distinct locations and viewing angles. These presets enable rapid, deterministic viewpoint changes while keeping the underlying environment fixed, allowing agents to survey a scene, zoom into regions of interest, or switch perspectives without introducing temporal inconsistency. This setup mirrors real PTZ deployment, where cameras can have set predefined viewpoints. Representative views from these environments are shown in Figure 3.

4.3 Ground Truth Annotation

To ground the tasks, we first created structured annotations for each camera preset. For every preset view, we documented observable entities and attributes (including counts and colors), spatial layout (e.g., left/right, near/far, and occlusions), OCR-relevant information, and how the scene would change under specific pan or tilt adjustments. We additionally recorded explicit absences (e.g., "no pedestrians or motorcycles in the parking lot"). These annotations serve as local ground truth: task prompts and expected answers were generated by an LLM conditioned on these annotations, ensuring that all tasks are anchored in verifiable observations rather than model-invented assumptions. Recording absences allowed an important extension–tasks whose correct outcome is the recognition that an object or attribute is not present.

4.4 Task Construction and Dataset Refinement

Task generation was seeded using a combination of the ground-truth annotations and a set of logged agent interaction traces from our physical PTZ deployment. These traces included full tool-call trajectories, intermediate observations, and operator feedback (e.g., thumbs up/down, some comments), which informed both representative user phrasing and expected behaviors. We additionally curated example task types for each evaluation category and provided guidance on a task distribution representative of realistic PTZ operation, spanning the categories summarized in Table 1. This process produced an initial draft of 541 tasks.

Each task specifies a question–answer pair and, where applicable, includes structured expectations about agent behavior, such as required tool calls (or call order) and observation scope (current-view vs. full-scene). Together, these specifications capture both realistic user queries and challenging cases that stress perception, tool routing, and multi-step reasoning, enabling systematic analysis of where agents succeed and where they fail.

We then refined the dataset through an iterative validation process involving multiple representative SLM–VLM pairings and an LLM-as-Judge. We systematically audited low-success rate tasks and manually inspected tasks and agent trajectories to correct impossible or underspecified questions, remove duplicates, and align expected answers and tool-call expectations with the intended execution standard. In parallel, we refined the judge's category-specific evaluation prompts to ensure that correctness judgments and error-mode attributions were consistent with our own assessments. Key adjustments included making open-ended prompts more permissive while retaining strict criteria for narrowly defined tasks.

Across three refinement passes, tasks and prompts were corrected, removed, or added as needed, the 536 tasks were finalized. The distribution of tasks can be seen in Table 1.

Table 1: Distribution of the 536 evaluation questions across categories. Tasks span visual perception (counting, descriptors, OCR, spatial reasoning) as well as control and reasoning skills (single calls, multi-step commands, and comparative queries).

Evaluation Category	Count	Evaluation Category	Count
Counting	95	Single Call	72
Descriptor	89	Multi-step Command	57
Location / Spatial	53	Multi-step Reasoning	54
OCR Identification	54	Comparative Relational	62

5 Evaluation Methodology

This section defines the evaluation methodology used to assess PTZ agent performance. We introduce the scoring protocol for determining correctness and attributing failure modes, and then describe the model pairing strategy used to compare planners and perception backbones.

5.1 Evaluation and Scoring Protocol

To support structured error analysis, we define a set of mutually exclusive error categories shown in Table 2. These categories capture failures arising from visual perception (e.g., misreading text or miscounting objects), reasoning errors, hallucinations, and tool-use mistakes such as missing calls, incorrect arguments, or improper scope selection.

Table 2: Error modes used in evaluation. When an answer is judged incorrect, the LM-as-Judge assigns one category to explain the failure.

Error mode	Description
VLM – Query	Incorrect or incomplete visual read (OCR or attributes).
VLM – Counting	Miscounts objects or misses occluded items.
Reasoning	Wrong conclusion despite correct tool outputs.
Hallucination	Claims facts not grounded in tool evidence.
Lack of Tool Call	Required tool never invoked.
Tool Args	Incorrect arguments or targets (wrong preset/object).
Scope	Wrong observation scope (*current* vs. *full*).
Tool Routing	Wrong tool(s) chosen or required tool omitted.

Table 3 lists the SLM planners and VLMs evaluated in this study. Model performance is assessed using an LLM-as-Judge framework that evaluates each agent execution holistically, rather than relying on deterministic string matching or rule-based checks. Such deterministic approaches break down for multi-step PTZ tasks, where multiple response phrasings, partial successes, or adaptive tool usage can all constitute valid outcomes. An LLM-based evaluator is therefore required to reliably assess both task completion and failure modes.

We use gpt-oss-120B as the sole LLM-as-Judge for all experiments [1]. For each agent execution, the judge is provided with the full trajectory—including tool calls, intermediate observations, and the final response—and produces a structured evaluation. Specifically, the judge returns (i) a binary correctness label and, for every incorrect execution, (ii) a single error attribution selected from the categories in Table 2, along with (iii) a short natural-language rationale explaining the decision. Accuracy throughout the paper is therefore defined as the fraction of executions judged correct under this rubric, while error analysis is grounded in the explicit failure modes defined above.

We selected gpt-oss-120B because it is among the strongest publicly available open-weight models at the time of evaluation and was trained explicitly for agentic reasoning and tool-use tasks [1]. Prior work and public benchmarks rank it competitively with leading proprietary judges: it performs comparably to GPT-4-class models on mixed-domain reasoning benchmarks [7, 18] and has been used successfully for evaluating agent behaviors in recent systems work [1, 11]. Using a single, fixed judge and rubric across all model pairings ensures consistency in relative comparisons, even though the procedure is inherently non-deterministic.

To improve reliability and alignment with human judgment, we adopted a human-in-the-loop evaluation process during benchmark construction and validation. This included manual inspection of judge rationales, iterative refinement of the scoring rubric, and review of full agent trajectories. These checks showed strong agreement between the judge's decisions and our own assessments. In practice, the judge consistently penalized smaller SLMs for missing tool calls, incorrect routing, or hallucinated answers, while larger planners were more often credited for partial or correct executions—patterns that align with our qualitative observations during development. While we do not claim a formal external human study,

Table 3: Models evaluated in our study. Top block lists SLM planners, bottom block lists VLMs. For MoE models, "Active Params" refers to the number of parameters active per token; dense models show full size. Precision indicates the primary runtime format used in evaluation. Notes summarize release details and intended role (baseline, quantized variant, or efficiency-focused).

Language Model	Total Size	Active Params	Precision	Notes
Qwen3-4B (Instruct-2507)	4B	4B	FP16	Smaller dense model [16]
Qwen3-4B (Instruct-2507-FP8)	4B	4B	FP8	Smaller dense model, quantized [16]
Qwen3-30B-A3B (Instruct-2507)	30B	3B	FP16	MoE, baseline [16]
Qwen3-30B-A3B (Instruct-2507-FP8)	30B	3B	FP8	MoE, quantized baseline [16]
Qwen3-32B (Instruct)	32B	32B	FP16	Dense baseline (older release) [16]
Qwen3-Next-80B-A3B (Instruct)	80B	3B	FP16	Latest MoE baseline; deeper, faster thinking [16]
Qwen3-Next-80B-A3B-FP8 (Instruct)	80B	3B	FP8	Quantized variant of Next-80B [16]
Vision–Language Model				
Moondream2 (release 2025-06-21)	2B	2B	FP16	Efficiency-focused VLM; "superword" tokenizer speeds answers by 20–40% without accuracy drop [12]
Moondream2-4bit (release 2025-04-14)	2B	2B	INT4	Quantization-aware trained; 42% less memory, 0.6% accuracy drop; 184 tok / s on RTX 3090 [12]
Moondream3 (preview 2025-09-18)	9B	2B	FP16	MoE VLM with 2B active experts; balances accuracy and efficiency [12]
Qwen2.5-VL-7B-Instruct	7B	7B	FP16	Full dense VLM; widely used [4]

this combination of a strong evaluator model, explicit rubric design, and extensive internal validation provides a robust and transparent basis for our evaluation.

5.2 Model Selection and Pairing Strategy

Our evaluation primarily pairs each SLM planner with the Moondream VLM family (Moondream2, Moondream2-4bit, and Moondream3), reflecting our deployment goal of edge-feasible perception. In our hardware setting, Moondream variants offered the most favorable latency–throughput trade-off while maintaining strong accuracy and clean JSON-based tool integration [12]. Moondream2 reports 89.55% accuracy on CountBench, while a comparable Qwen2.5-VL 72B model reports 93.6% [4, 12, 14]. Motivated by this gap, we include a targeted cross-family comparison pairing our largest planner, Qwen3-Next-80B-A3B, with Qwen2.5-VL-7B to examine whether benchmark trends and shared model lineage translate to downstream PTZ tasks. Due to the substantial GPU requirements of the VLM, this configuration was evaluated once as a controlled comparison.

6 Results

6.1 Where Errors Come From: A Decomposition

Results in Table 4 reveal a structured error profile in language-driven PTZ agents, with distinct failure modes emerging from planner capacity and visual perception. Accuracy on visually demanding categories such as Counting (52.6–76.8%) and OCR Identification (38.9–83.3%) is substantially lower than on tool-centric categories like Single Call (97.2–100.0%) and Multi-step Command (86.0–98.2%), indicating that perception-heavy tasks are inherently more challenging. However, these aggregate trends mask important differences across planner capacity.

Figure 4 clarifies how failures are distributed across model pairings. For weaker SLM planners, errors frequently arise from hallucinations or improper tool usage, including argument-level mistakes such as issuing imprecise spatial references (e.g., confusing "car" with "car to the right"). As planner capacity increases, these failure modes diminish markedly: stronger SLMs are more reliable, better grounded in tool outputs, and more consistent in invoking the correct tools over the correct observation scope. Tool-routing and scope-selection errors are rare for these planners, suggesting diminishing returns from further SLM scaling.

Among all combinations, failures are dominated by VLM querying and counting errors. For stronger SLM pairings, perception-related errors constitute the largest share of remaining failures, confirming that visual understanding becomes the primary limiter of end-to-end performance in the high-capacity planner regime. This shift highlights a transition from planner-limited to perception-limited behavior as SLM capacity increases.

Disentangling planner and perception effects further reinforces this asymmetry. Holding the VLM family fixed, changing only the SLM yields a substantial accuracy spread of 5.6–12.4%, reflecting the importance of planner capacity for reducing hallucinations and reasoning failures. In contrast, the largest accuracy spread across VLM backbones is just 6.9%. Notably, OCR and fine-grained counting tasks show the largest gains between Moondream2 and Moondream3, indicating that modular improvements to the vision backbone do translate into measurable gains.

To isolate the impact of perception more directly, we re-compute task accuracy by treating failures attributed to VLM counting and query errors as correct. Under this oracle setting, average accuracy increases from 66.2% to 82.0% (+15.8%), indicating that perception failures account for 46.7% of errors on average. Among the strongest SLM planners—where hallucination, routing, and reasoning errors are least frequent—perception constitutes an even larger fraction of remaining failures, confirming that the vision backbone ultimately sets the performance ceiling for PTZ agents.

6.2 Scope Sensitivity

Query scope interacts strongly with perception difficulty. Current-view queries achieve 66.9% average accuracy, compared to 58.3% for full-scene queries. Given that scope-selection errors are rare, this gap likely reflects VLMs suffering from accuracy on panoramas, particularly on counting and text recognition tasks.

Table 4: Evaluation-category accuracy (%) for each language model (rows), grouped by paired VLM family (blocks). Within each family, SLM planners are ordered by parameter scale (4B → 30B-A3B → 32B → 80B-A3B), with FP16 results listed before their quantized variants. Each cell reports accuracy (%) for that category. "Average" denotes the unweighted mean accuracy across categories for each row. **Bold** indicates the highest score within a VLM family for a given category, while underlined indicates the highest score across all families.

Model	Comparative Relational	Counting	Descriptor	Location / Spatial	Multi-step Command	Multi-step Reasoning	OCR Identification	Single Call	Average
Moondream2									
Qwen3-4B	27.4	68.4	38.2	52.8	89.5	64.8	38.9	97.2	59.7
Qwen3-4B-FP8	27.4	69.5	40.4	56.6	89.5	61.1	40.7	97.2	60.3
Qwen3-30B-A3B	43.5	70.5	44.9	52.8	93.0	81.5	53.7	**98.6**	67.3
Qwen3-30B-A3B-FP8	**50.0**	72.6	**50.6**	56.6	89.5	79.6	**59.3**	**98.6**	69.6
Qwen3-32B	41.9	70.5	46.1	**64.2**	**96.5**	72.2	53.7	**98.6**	68.0
Qwen3-Next-80B-A3B	46.8	**76.8**	48.3	58.5	**96.5**	**83.3**	55.6	**98.6**	**70.6**
Moondream2-4bit									
Qwen3-4B	32.3	62.1	47.2	50.9	91.2	66.7	48.1	**100.0**	62.3
Qwen3-4B-FP8	27.4	62.1	48.3	52.8	89.5	66.7	46.3	97.2	61.3
Qwen3-30B-A3B	41.9	64.2	49.4	58.5	**98.2**	72.2	**51.9**	98.6	**66.9**
Qwen3-30B-A3B-FP8	**50.0**	**65.3**	55.1	50.9	93.0	72.2	40.7	98.6	65.7
Qwen3-32B	37.1	60.0	**59.6**	**64.2**	87.7	68.5	**51.9**	98.6	65.9
Qwen3-Next-80B-A3B	36.5	**65.3**	52.8	58.5	**98.2**	**75.9**	48.1	98.6	66.8
Moondream3									
Qwen3-4B	32.3	68.4	33.7	47.2	87.7	55.6	66.7	**100.0**	61.4
Qwen3-4B-FP8	30.6	73.7	34.8	58.5	87.7	59.3	57.4	98.6	62.6
Qwen3-30B-A3B	**56.5**	**76.8**	37.1	**66.0**	**98.2**	**79.6**	77.8	98.6	**73.8**
Qwen3-30B-A3B-FP8	48.4	74.7	38.2	52.8	86.0	75.9	77.8	98.6	69.1
Qwen3-32B	45.2	68.4	**39.3**	52.8	89.5	72.2	**83.3**	**100.0**	68.8
Qwen3-Next-80B-A3B	45.2	72.6	34.8	47.2	**98.2**	74.1	**83.3**	98.6	69.3
Qwen2.5-VL-7B									
Qwen3-Next-80B-A3B	**33.9**	**52.6**	**52.8**	**66.0**	**93.0**	**72.2**	**75.9**	**100.0**	**68.3**

Table 5: Median inference latency (s) and mean tool calls across all SLM and VLM evaluations.

Model / Setting	Latency (s)	Tool Calls
Vision-Language Models		
Qwen2.5-VL-7B	5.46	1.34
Moondream2	5.96	1.43
Moondream2-4bit	6.87	1.45
Moondream3	9.18	1.50
Current view → Panorama	6.07 → 7.43	—
Small Language Models		
Qwen3-4B	1.94	1.44
Qwen3-4B-FP8	0.96	1.41
Qwen3-30B-A3B	1.39	1.61
Qwen3-30B-A3B-FP8	2.62	1.54
Qwen3-32B	3.00	1.51
Qwen3-Next-80B-A3B	1.42	1.63

6.3 Latency

Table 5 shows that end-to-end latency is dominated by perception rather than planning. VLM backbones vary widely in per-call inference cost, and expanding the observation scope to a full panorama increases latency by 22.4% due to larger image inputs. In contrast, planner latency remains comparatively stable across configurations: MoE planners achieve inference times close to those of small dense models despite much larger total capacity. Quantization plays a meaningful role in deployment, but its impact on speed is configuration-dependent in our setup, improving latency in some cases while degrading it in others, likely due to hardware, kernel, and serving-path effects. Taken together, these results indicate that perception and observation scope set the dominant latency costs, while planner choice affects both accuracy and systems cost, and can improve accuracy without incurring proportional latency increases (as with MoE planners).

6.4 Architecture Selection

Across VLM families, MoE SLMs achieve the highest accuracies while maintaining inference times comparable to dense planners. In practice, this indicates that MoE planners make more effective tool-selection and sequencing decisions without incurring a meaningful latency penalty. Quantization likewise preserves accuracy—performance differences are small and sometimes favor the quantized variant. Together, these results support MoE planners and quantization as practical deployment strategies for real-time PTZ agents.

Consistent with these trends, the strongest SLM–VLM pairings combine capable MoE planners with the most accurate available

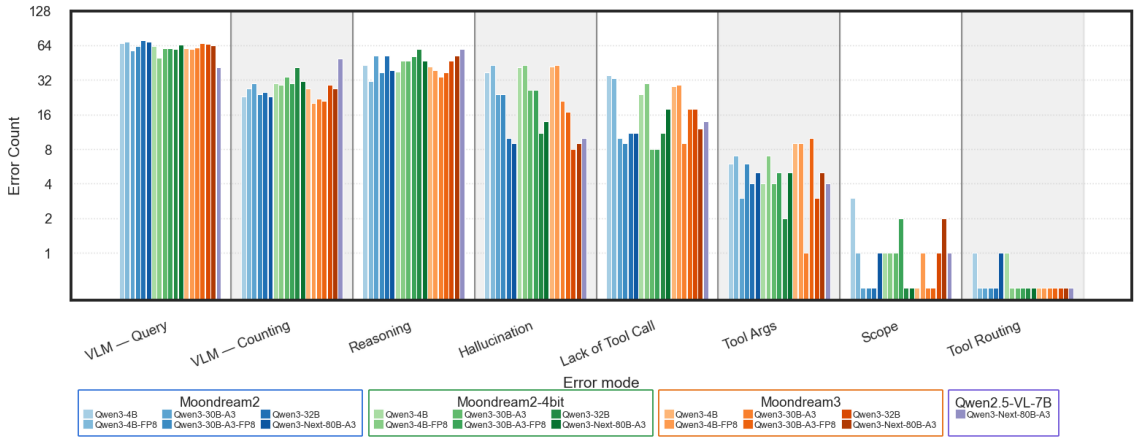

Figure 4: Error mode distribution across SLM–VLM combinations. Colors denote VLM selection, with shade intensity indicating increasing SLM size. Shaded bands and vertical dividers separate error categories.

vision backbones. The best-performing configuration, Moondream3 paired with Qwen3-30B-A3B, achieved 73.8% task success.

7 Limitations

Because the benchmark is grounded in structured, worksheet-based prompts, queries are more constrained and less ambiguous than those encountered in real deployments. As a result, certain failure modes—such as incorrect tool sequencing, improper command scope, or recovery from ambiguous instructions—appear less frequently in aggregate metrics than they would in the field.

Additionally, latency measurements were likely skewed by differences in serving configurations rather than intrinsic model speed alone. Qwen SLM/VLM models were served using optimized vLLM-based infrastructure, whereas Moondream was evaluated in a non-optimized self-hosted setup. In some evaluation runs, multiple Moondream runs shared the same hardware, reducing effective throughput. When evaluated in isolation without resource contention, Moondream models consistently achieved lower, real-time per-call latency than the reported averages suggest.

Finally, we did not benchmark closed frontier models. While such systems may offer higher raw accuracy, their reliance on remote inference introduces network and queuing latency that is incompatible with real-time, language-guided PTZ control. Our evaluation therefore focuses on models that are realistically deployable in low-latency edge settings, and excludes black-box API models whose deployment characteristics fall outside the scope of this work.

8 Discussion and Future Work

Our findings show that compact MoE and quantized architectures provide the best accuracy–latency balance for real-time PTZ control. Dense 32B planners are impractically slow, while 30B MoE models—activating only 3B parameters—outperform smaller dense planners in reasoning. Quantization preserves accuracy but introduces A100-specific latency penalties due to kernel and serving-path limitations, underscoring that deployment performance depends on hardware and software support as much as model architecture. Scaling planners beyond 30B-A3B yields no clear accuracy gains at substantially higher memory cost, making 30B-A3B the most favorable deployment point.

While planner capacity is largely sufficient at this scale, residual planner-side gaps remain. Even strong SLMs occasionally fail to challenge ambiguous queries, reflect on prior tool outputs, or refine visual observations through narrower scope or second-pass queries. These issues—imprecise tool arguments, limited self-checking, and shallow use of conversational context—point to opportunities for improvement via prompt refinement, lightweight reflection, or RL that rewards corrective tool use.

Beyond these planner limitations, perception emerges as the dominant ceiling on end-to-end performance. Routing and sequencing errors are rare among MoE planners, but accuracy remains constrained by VLM failures in tasks such as OCR and fine-grained counting. This suggests future accuracy gains will hinge on stronger VLMs. Our latency results also highlight that for constrained subtasks (e.g., tracking), specialized detectors on lighter architectures such as YOLO may remain indispensable, pointing to hybrid stacks where the SLM orchestrates when and how to call them. Together, these findings validate our modular design: separating planning and perception makes real-time edge deployment practical today.

Future work will improve planner reliability via prompt refinement and reinforcement learning, expand SCOPE's use of dynamic scene injection for tracking, and extend the framework to multi-camera, multi-agent deployments for cooperative, full-perimeter perception.

References

[1] Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K. Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. 2025. gpt-oss-120b & gpt-oss-20b Model Card. arXiv:2508.10925 [cs.CL] https://arxiv.org/abs/2508.10925

[2] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. 2022. Do as I can, not as I say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691* (2022).

[3] Peter Anderson, Qi Wu, Damien Teney, Jake Bruce, Mark Johnson, Niko Sünderhauf, Ian Reid, Stephen Gould, and Anton Van Den Hengel. 2018. Vision-and-Language Navigation: Interpreting Visually-Grounded Navigation Instructions

in Real Environments. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 3674–3683.

[4] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. 2025. Qwen2.5-VL Technical Report. *arXiv preprint arXiv:2502.13923* (2025).

[5] Lee Boonstra. 2024. *Prompt Engineering*. Whitepaper. Google Cloud. https://www.kaggle.com/whitepaper-prompt-engineering

[6] Alexiy Buynitsky, Sina Ehsani, Bhanu Pallakonda, and Pragyana Mishra. 2025. Camera Control at the Edge with Language Models for Scene Understanding. *arXiv preprint arXiv:2505.06402* (2025).

[7] Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Banghua Zhu, Hao Zhang, Michael Jordan, Joseph E Gonzalez, et al. 2024. Chatbot arena: An open platform for evaluating llms by human preference. In *Forty-first International Conference on Machine Learning*.

[8] Abhishek Das, Samyak Datta, Georgia Gkioxari, Stefan Lee, Devi Parikh, and Dhruv Batra. 2018. Embodied question answering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, Salt Lake City, UT, USA, 1–10.

[9] Matt Deitke, Christopher Clark, Sangho Lee, Rohun Tripathi, Yue Yang, Jae Sung Park, Mohammadreza Salehi, Niklas Muennighoff, Kyle Lo, Luca Soldaini, et al. 2025. Molmo and Pixmo: Open Weights and Open Data for State-of-the-Art Vision–Language Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 91–104.

[10] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300* (2020).

[11] Brad Hilton, Kyle Corbitt, David Corbitt, Saumya Gandhi, Angky William, Bohdan Kovalenskyi, and Andie Jones. 2025. ART: Agent Reinforcement Trainer. https://github.com/openpipe/art.

[12] Vikhyat Karamcheti. 2025. Moondream: Lightweight Vision–Language Models (Moondream2, Moondream2-4bit, Moondream3-preview). https://huggingface.co/moondream. Includes Moondream2 (2025-06-21), Moondream2-4bit (2025-04-14), Moondream3-preview (2025-09-18).

[13] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. 2022. Code as policies: Language model programs for embodied control. *arXiv preprint arXiv:2209.07753* (2022).

[14] Roni Paiss, Ariel Ephrat, Omer Tov, Shiran Zada, Inbar Mosseri, Michal Irani, and Tali Dekel. 2023. Teaching CLIP to Count to Ten. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, 3170–3180. https://openaccess.thecvf.com/content/ICCV2023/html/Paiss_Teaching_CLIP_to_Count_to_Ten_ICCV_2023_paper.html

[15] Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, Vancouver, Canada. https://openreview.net/forum?id=2GmDdhBdDk ICML 2025 poster.

[16] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388

[17] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. 2016. You Only Look Once: Unified, Real-Time Object Detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 779–788.

[18] Nidhish Shah, Zulkuf Genc, and Dogu Araci. 2024. StackEval: Benchmarking LLMs in Coding Assistance. In *Advances in Neural Information Processing Systems*, Vol. 37.

[19] Mohit Shridhar, Jesse Thomason, Daniel Gordon, Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke Zettlemoyer, and Dieter Fox. 2020. ALFRED: A benchmark for interpreting grounded instructions for everyday tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE/CVF, 10740–10749.

[20] Sai H Vemprala, Rogerio Bonatti, Arthur Bucker, and Ashish Kapoor. 2024. ChatGPT for robotics: Design principles and model abilities. *IEEE Access* 12 (2024), 55682–55696.

[21] Junyang Wang, Haiyang Xu, Jiabo Ye, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. 2024. Mobile-agent: Autonomous multi-modal mobile device agent with visual perception. *arXiv preprint arXiv:2401.16158* (2024).

[22] Xiang Yue, Tianyu Zheng, Yuansheng Ni, Yubo Wang, Kai Zhang, Shengbang Tong, Yuxuan Sun, Botao Yu, Ge Zhang, Huan Sun, Yu Su, Wenhu Chen, and Graham Neubig. 2024. MMMU-Pro: A More Robust Multi-discipline Multimodal Understanding Benchmark. *arXiv preprint arXiv:2409.02813* (2024).